

LATENT PLANNING VIA EMBEDDING ARITHMETIC: A CONTRASTIVE APPROACH TO STRATEGIC REASONING

Anonymous authors
Anonymous submission

ABSTRACT

This paper asks whether planning can be made explicit as geometry in a learned representation, rather than as a policy head, value head, or learned dynamics model. Our contribution is SOLIS, a chess engine that learns an evaluation-aligned embedding with supervised contrastive learning and then plans inside that space. The learned geometry supports two linked operations: proximity groups positions with similar outcomes, and a single global *advantage vector* points from losing regions toward winning regions. At inference time, SOLIS ranks legal continuations by alignment with this direction, turning action selection into lightweight vector arithmetic plus shallow search. In chess, this design reaches competitive strength under constrained search, including 2500+ Elo at 5-ply search under 50 ms/move conditions. The results position evaluation-aligned latent planning as a simple, interpretable alternative to approaches that require explicit dynamics learning or policy/value training.

1 INTRODUCTION

Planning in high-dimensional state spaces remains difficult because useful decisions must be selected from rapidly expanding action sequences. Standard reinforcement learning approaches often rely on broad search or learned policies, both of which can become inefficient as the action space grows (Jiang et al., 2023; Gieslmann and Pokorný, 2022). Latent planning offers one route around this bottleneck by reasoning over compact representations rather than raw states (Hafner et al., 2019; 2020; Schrittwieser et al., 2020). The contribution we pursue is more specific: we test whether the representation itself can carry enough evaluation structure to guide search.

We introduce **SOLIS** (*Strategic Optimization via Latent Interpolative Spaces*) to make this question concrete. SOLIS is a chess engine whose transformer encoder is trained with supervised contrastive learning so that positions with similar Stockfish-evaluated outcomes occupy nearby regions of a latent space. This gives the paper its first main contribution: an evaluation-aligned representation in which chess advantage is encoded geometrically. Its second contribution is an inference rule that uses this geometry directly: SOLIS ranks candidate moves by their alignment with a single global “advantage vector” from losing regions toward winning regions.

The empirical contribution is to show that this simple latent planner remains effective under restricted search. With only a 5-ply search under 50 ms/move blitz conditions, SOLIS achieves an Elo of 2500+. In equal-depth matches at search depth $S \in \{3, 4, 5\}$ with a fixed branching factor, it is competitive with a configured Stockfish baseline in our setup. We also show that real game trajectories trace coherent paths through the embedding space, making the learned planning signal easier to inspect qualitatively.

Together, these results distinguish SOLIS from latent planning systems that combine learned dynamics with policy or value heads (Hafner et al., 2019; 2020; Schrittwieser et al., 2020). The central claim is deliberately narrower and reviewer-facing: when evaluations are available or learnable, a contrastive embedding plus a global advantage direction can provide a low-overhead planning mechanism. Chess is our testbed, but the same contribution may be relevant to other perfect-information domains with large action spaces.

2 BACKGROUND

This section positions SOLIS relative to three ingredients needed to understand the contribution: sequential decision making, chess-engine search, and contrastive representation learning. The common thread is the search bottleneck: action sequences grow exponentially with depth, so the quality of the representation used to rank continuations can matter as much as the amount of expansion. We use this background to clarify what SOLIS keeps from prior planning systems and what it removes.

2.1 MARKOV DECISION PROCESSES

Sequential decision-making tasks are commonly formalized as Markov Decision Processes (MDPs), represented by the tuple $\{\mathcal{S}, \mathcal{A}, T, R, p(s_0), \gamma\}$. Here $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, T is the transition model, R is the reward function, $p(s_0)$ is the initial state distribution, and $\gamma \in [0, 1]$ discounts future rewards (Puterman, 2014). A policy $\pi : \mathcal{S} \rightarrow p(\mathcal{A})$ maps each state to a distribution over actions. This notation highlights the design choice made by SOLIS: it does not learn a policy over $\mathcal{A}$, but instead learns a state representation used to rank actions during search.

Within an MDP, a *model* provides reversible access to the dynamics: given a state–action pair, it returns the next state and reward (Moerland et al., 2022). Board games such as chess and Go supply this model through their rules, whereas domains without known rules require the model to be learned from data (Gieslmann and Pokorny, 2022). SOLIS uses the known chess transition function for legal move generation, but its distinctive component is not a dynamics model; it is the evaluation-aligned geometry used to choose which continuations to keep.

Planning and reinforcement learning differ in how they use access to dynamics (Moerland et al., 2022). Planning methods simulate from the current state to select a local action, resembling mental lookahead (Russell and Norvig, 1995); after the action is chosen, the local computation is discarded. Reinforcement learning methods, by contrast, typically interact with the environment step by step without reversible dynamics access (Sutton and Barto, 1998). They therefore learn a *global solution*, such as a policy or value function, that generalizes across states. SOLIS occupies a middle ground: it performs local lookahead, but the criterion used to rank local continuations comes from a globally trained representation.

Model-based reinforcement learning combines these ideas by using a model while also learning a global object. Following Moerland et al. (2022), one case learns both the dynamics and the global solution, as in Dyna (Sutton, 1991). A second case assumes known dynamics, as in AlphaZero (Silver et al., 2017), and learns the global policy or value components. This distinction frames the contribution of SOLIS: known game rules support search, but the learned object is neither a policy nor a value head; it is a contrastive state embedding.

2.1.1 MODEL-BASED RL WITH A LEARNED MODEL

A large body of model-based RL learns both dynamics and a global decision rule. PlaNet (Hafner et al., 2019) learns a latent dynamics model and combines it with a learned global solution. Dreamer (Hafner et al., 2020) and its successors DreamerV2 (Hafner et al., 2022) and DreamerV3 (Hafner et al., 2024) scale this recipe to more complex tasks while retaining global value functions and policies. MuZero (Schrittwieser et al., 2020) also learns latent dynamics, then strengthens action selection with tree search while training policy and value functions. Dyna (Sutton, 1991) and MBPO (Janner et al., 2021) similarly use learned models to support value or policy learning. SOLIS is meant to isolate a different contribution: planning from an evaluation-aligned representation without learning latent dynamics.

2.1.2 MODEL-BASED RL WITH A KNOWN MODEL

When dynamics are available, the learning problem shifts toward how search should be guided. AlphaZero (Silver et al., 2017) uses exact game rules as the model and combines Monte Carlo tree search (MCTS) with learned policy and value heads. SAVE (Hamrick et al., 2020) similarly assumes a simulator and integrates Q-learning with MCTS: a learned Q-function provides a search prior, and Q-estimates produced by search are amortized back into the network. SOLIS keeps the known-

model setting but removes the learned policy/value interface, replacing it with geometric ranking in the embedding space.

2.1.3 LATENT PLANNING

Latent planning reduces the burden of decision making by operating in a compressed space rather than directly over raw states. PlaNet (Hafner et al., 2019), Dreamer (Hafner et al., 2020), and MuZero (Schrittwieser et al., 2020) pair latent dynamics with value or policy heads so that simulations can occur in the learned space. Expansive Latent Space Trees (ELAST) (Gieselmann and Pokorny, 2022) use contrastive representations with sampling-based expansion for long-horizon control. Trajectory Autoencoding Planner (TAP) (Jiang et al., 2023) learns a compact latent action space and plans over discrete action codes. These systems motivate our setting while also clarifying the gap SOLIS targets: whether an evaluation-aligned state embedding alone can guide action ranking.

SOLIS therefore contributes a deliberately stripped-down latent planning mechanism. It does not train policies or value heads, and it does not learn a dynamics model. Instead, it distills evaluation targets into the representation and uses the resulting geometry at inference, so planning becomes movement between regions associated with favorable outcomes.

2.2 CHESS ENGINES

Chess engines provide a sharp testbed because the rules are known, the action space is structured, and search efficiency is measurable. IBM’s Deep Blue demonstrated the power of scaled chess computation in 1997 by defeating world champion Garry Kasparov. Early Stockfish versions (Romstad et al., 2008) used a related recipe: handcrafted evaluation functions combined with deep alpha-beta tree search. That recipe achieved superhuman performance, but its strength depended heavily on the chess expertise embedded in the static evaluator.

The limitations of static evaluation motivated engines that learn evaluation from data rather than from hand-coded features. AlphaZero (Silver et al., 2017) and Leela Chess Zero (Pascutto and Linscott, 2018) showed that self-play reinforcement learning can produce strong evaluators using only the rules of chess. Modern Stockfish also adopted an efficiently updatable neural network evaluation module (NNUE) (Nasu, 2018), improving position evaluation while retaining traditional alpha-beta search. SOLIS builds on this shift toward learned evaluation, but asks whether the learned evaluator can be organized as a latent space for planning.

Despite large gains in playing strength, the core search template still often traces back to ideas from v. Neumann (1928) and Shannon (1950): expand many states, prune aggressively, and evaluate leaves (Lai, 2015). The chess-specific contribution of this paper is to test a different search interface. Rather than relying primarily on broad expansion, SOLIS uses contrastive learning to make high-quality continuations salient in a high-dimensional embedding space.

2.3 CONTRASTIVE LEARNING

Contrastive learning is the representation-learning tool that makes SOLIS possible. It trains an encoder to place similar inputs near one another and dissimilar inputs farther apart in latent space (Chen et al., 2020). Given a batch I containing anchor input x_i , an encoder $f(\cdot)$ maps the anchor to a normalized embedding $z_i = f(x_i)$. Positive pairs (z_i, z_j) encode the notion of similarity; all other in-batch samples serve as negatives. In SOLIS, the key design choice is to define similarity by evaluation proximity rather than by data augmentation alone.

A common contrastive objective is InfoNCE (van den Oord et al., 2019), defined as:

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \in A(i)} \exp(\text{sim}(z_i, z_k)/\tau)}, \quad (1)$$

where $A(i) = I \setminus \{i\}$ is the set of all non-anchor samples in the batch, $\text{sim}(\cdot, \cdot)$ is the similarity metric, and τ is the temperature parameter. In self-supervised contrastive learning, z_j is typically an augmentation of z_i , so labels are unnecessary (Chen et al., 2020; Khosla et al., 2021). This objective supplies the basic geometry, but SOLIS requires labels so that geometry corresponds to chess advantage.

When labels are available, the supervised contrastive loss (SupCon) (Khosla et al., 2021) extends InfoNCE by allowing multiple positive examples per anchor. The SupCon loss is defined as:

$$\mathcal{L}_{\text{sup}} = \sum_{i \in I} \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\text{sim}(z_i, z_p)/\tau)}{\sum_{a \in A(i)} \exp(\text{sim}(z_i, z_a)/\tau)}, \quad (2)$$

where $P(i)$ is the set of positives for anchor i , and $A(i)$ is defined as above. Increasing the number of positive and negative pairs can improve representation quality (Khosla et al., 2021; Chowdhury et al., 2024). For SOLIS, this supervised form is central because it lets Stockfish win probabilities define which positions should share a neighborhood.

3 METHODS

We now make the contribution operational. The method has four parts: FEN tokenization, a transformer encoder, supervised contrastive training on evaluation labels, and embedding-guided action selection. The first three steps create the evaluation-aligned space; the final step shows how SOLIS plans using that space without a policy head, value head, or learned dynamics model.

3.1 INPUT REPRESENTATION AND TOKENIZATION

SOLIS represents each chess position by its Forsyth-Edwards Notation (FEN) ASCII string, which encodes piece placement, side to move, castling rights, en passant target, half-move clock, and full-move counter. Following Ruoss et al. (2024), we expand run-length encodings and tokenize each FEN into a fixed-length sequence of 77 tokens. This compact input format supports efficient embedding, but it also marks an important limitation of the contribution: as noted by Ruoss et al. (2024), omitting full move history makes the resulting chess state non-Markovian for rules such as threefold repetition and the fifty-move rule. We return to this limitation in Section 5.

3.2 ENCODER ARCHITECTURE

The encoder contribution is intentionally simple: we adapt a multi-layer transformer (Vaswani et al., 2023) to tokenized chess positions and use its output as the latent state. We train two sizes, SOLIS Base and SOLIS Mini, to test whether the planning mechanism remains useful across representation capacity. Table 1 summarizes the configurations used in the experiments.

Table 1: SOLIS encoder configurations. Mini and Base differ mainly in embedding dimension and parameter count, allowing us to test whether the latent-planning contribution scales with representation capacity.

Model	Layers	Embedding dim. D	Heads	MLP Size	Params
SOLIS Mini	6	128	8	256	0.8M
SOLIS Base	6	1024	16	1024	41M

Each tokenized position is mapped to D -dimensional vectors with a learned token embedding matrix. We prepend a classification token (CLS) (Devlin et al., 2019; Dosovitskiy et al., 2021) so the encoder can aggregate board-level information, and because the sequences have fixed length, we add learned positional encodings (Ruoss et al., 2024). Stacked transformer layers with GELU (Hendrycks and Gimpel, 2023) activations and dropout (Srivastava et al., 2014) process the sequence. The final CLS hidden state is then linearly projected and ℓ_2 normalized, producing the embedding on which all subsequent planning operations are performed.

3.3 SUPERVISED CONTRASTIVE TRAINING

The training contribution is to make evaluation labels define the latent geometry. We train the encoder on 5 million randomly sampled ChessBench positions (Ruoss et al., 2024), each annotated with a Stockfish-evaluated win probability for the player to move. We normalize every label to

White’s perspective, so values near 1.0 indicate a decisive White advantage and values near 0.0 indicate a Black advantage.

Positive pairs are defined by evaluation proximity. We set δ as the margin for treating two positions as similar; for each anchor, we precompute positions whose win probabilities differ by less than δ and randomly sample five positives during training. The batch-level mask applies the same rule by marking pairs as positive when their evaluation difference is below δ . This construction pulls together positions with nearly equal outcomes and separates positions with larger evaluation gaps, creating the ordered latent structure that SOLIS later uses for planning.

We optimize this geometry with supervised contrastive learning (SupCon) (Khosla et al., 2021) using the constructed masks for positives and negatives. Pairwise scores use cosine similarity, and the temperature $\tau = 0.07$ (Girdhar et al., 2023; Radford et al., 2021) controls distribution sharpness. We train with stochastic gradient descent, momentum 0.9 (Dosovitskiy et al., 2021; Chen et al., 2020; He et al., 2015), batch size 128, for 400,000 steps on six NVIDIA L40S GPUs (48 GB each) with data parallelism (Paszke et al., 2019). Appendix A.2 gives the full hyperparameter specification.

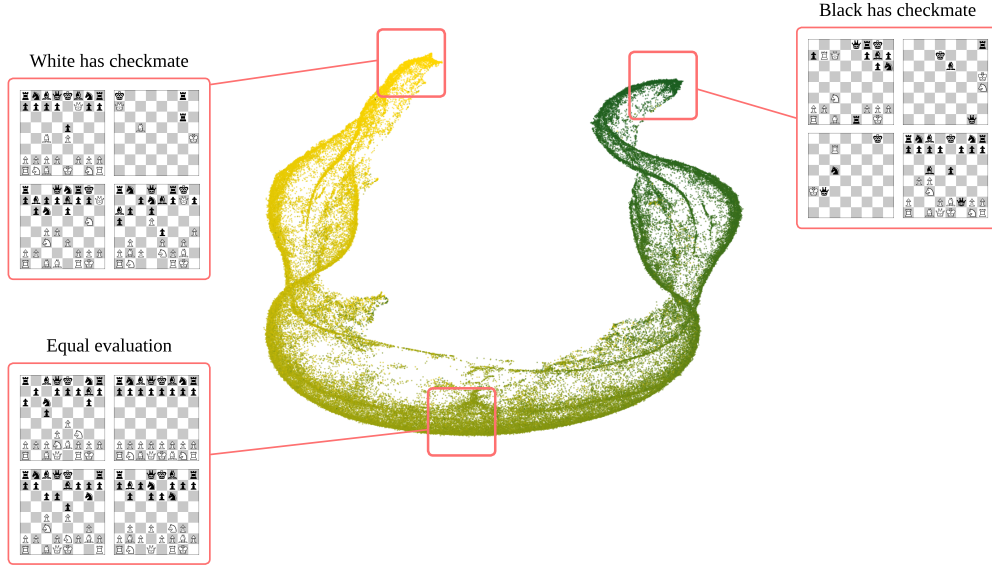


Figure 1: Evaluation-aligned geometry learned by SOLIS Base. The UMAP projection is colored by White win probability (gold = White favored, green = Black favored), showing the representation structure that later supports advantage-vector planning.

Figure 1 provides a qualitative check that the training objective produces the intended contribution: an evaluation-organized space. We project SOLIS Base embeddings to 2D with UMAP (McInnes et al., 2020); each point is a held-out chess position colored by its Stockfish-evaluated White win probability. Strong White positions appear in gold (*White has checkmate*), while strong Black positions appear in dark green (*Black has checkmate*), revealing a smooth transition that motivates the global advantage direction.

3.4 EMBEDDING-GUIDED SEARCH

At inference, SOLIS turns the learned geometry into a planner by ranking legal continuations along one global advantage direction. We estimate this direction from held-out embeddings at the two evaluation extremes,

$$\mu_{\text{White}} = \mathbb{E}[z \mid p=1.0], \quad \mu_{\text{Black}} = \mathbb{E}[z \mid p=0.0],$$

and define the normalized advantage vector

$$\vec{a} = \frac{\mu_{\text{White}} - \mu_{\text{Black}}}{\|\mu_{\text{White}} - \mu_{\text{Black}}\|}.$$

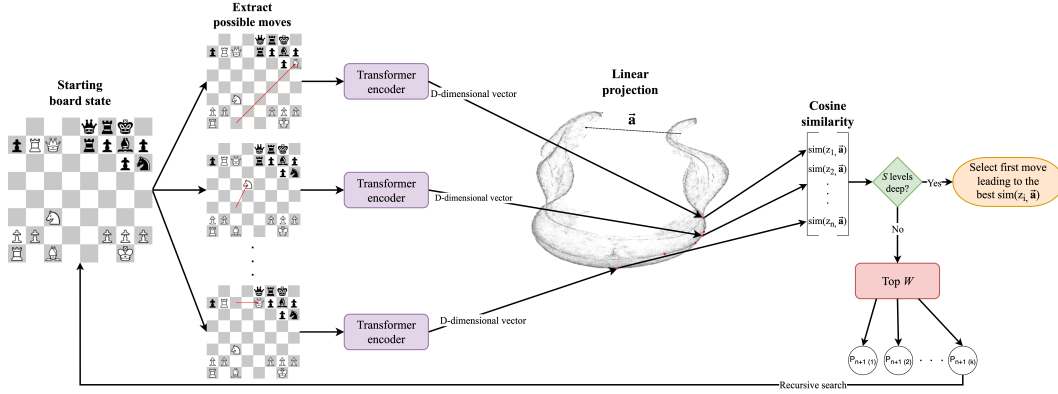


Figure 2: SOLIS planning pipeline. Candidate moves are embedded, scored by alignment with the global advantage vector, pruned to the top- W continuations, and searched recursively. For simplicity, the diagram shows the unanchored score $\cos(z', \vec{a})$.

We evaluate two scalar scoring rules for this same contribution: both use the advantage vector, but they differ in whether the projection is measured from the origin or from the Black extreme.

3.4.1 UNANCHORED (GLOBAL-DIRECTIONAL)

The unanchored rule is the most direct version of latent planning by embedding arithmetic. It scores each child embedding z' by raw alignment with the advantage direction,

$$\text{score}_{\text{dir}}(z') = \langle z', \vec{a} \rangle,$$

This rule favors moves whose embeddings extend furthest along the advantage direction, independent of the origin. Figure 2 illustrates this scoring mechanism.

3.4.2 ANCHORED (RELATIVE PROJECTION)

The anchored rule makes the same planning signal relative to the Black endpoint by shifting each child embedding before projection:

$$\text{score}_{\text{anch}}(z') = \langle z' - \mu_{\text{Black}}, \vec{a} \rangle,$$

The resulting value is the signed distance from μ_{Black} along $\vec{a}$. This removes dependence on an arbitrary origin and, in our experiments, gives a more stable basis for comparing continuations.

Both scoring mechanisms use the same shallow search procedure, summarized in Figure 2. Starting from the “Starting board state”, SOLIS “Extract possible moves”, embeds each child in parallel with the “Transformer encoder”, and computes either $\text{score}_{\text{dir}}$ or $\text{score}_{\text{anch}}$ in the “Cosine similarity” block. It then keeps the “Top- W ” children. If “ S levels deep?” is satisfied, the system terminates and “Select first move...”; otherwise, it continues “Recursive search” with min-max preference, where White maximizes and Black minimizes. We use Zobrist hashing (Zobrist, 1990) for a transposition table so repeated positions are not re-evaluated.

4 RESULTS

4.1 RATING CALCULATION

The first experimental question is whether evaluation-aligned latent planning produces measurable playing strength as search depth and width vary. We estimate Elo by evaluating each model against Stockfish with a fixed per-move time limit of 50 ms (Ruoss et al., 2024); Appendix A.3 gives the Stockfish configuration. For each depth/width setting, SOLIS plays 600+ games with alternating colors against Stockfish at multiple Elo caps. We evaluate both unanchored and anchored planning, choosing caps so that each SOLIS configuration has at least one positive-win-rate and one negative-win-rate matchup. We estimate Elo ratings and 95% CIs with Bayesian logistic regression using

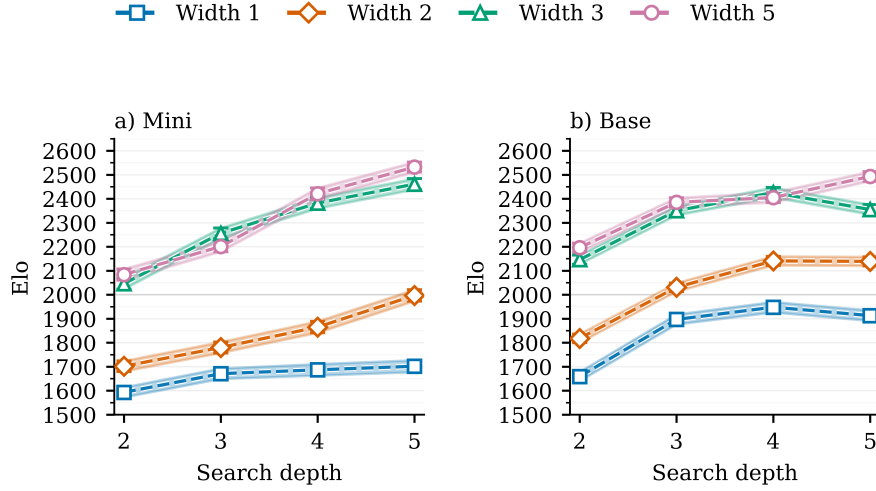


Figure 3: Elo scaling for SOLIS Mini and Base with *unanchored* advantage-vector planning. The curves show how the latent planner responds to search width and depth; shaded bands denote 95% confidence intervals.

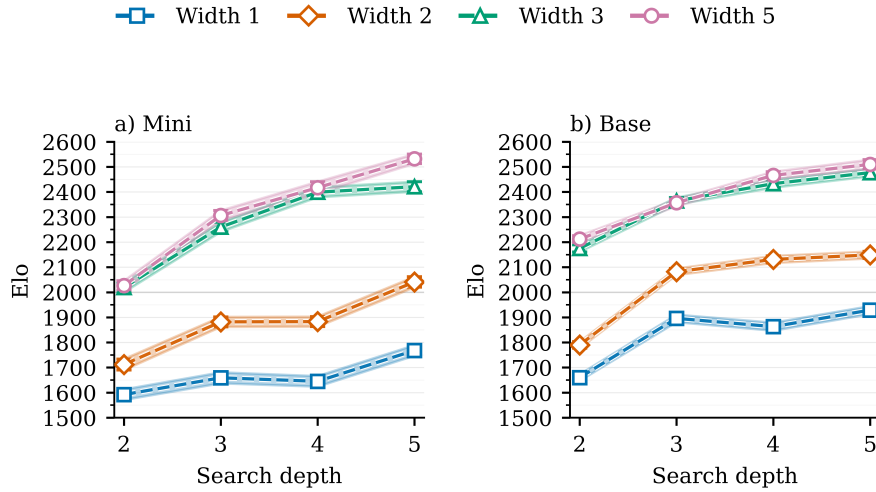


Figure 4: Elo scaling for SOLIS Mini and Base with *anchored* advantage-vector planning. Anchoring measures progress from the Black endpoint of the embedding space; shaded bands denote 95% confidence intervals.

BayesElo (Coulom, 2008) and its default confidence parameter of 0.5. Figure 3 reports unanchored results, Figure 4 reports anchored results, and Appendix A.6 provides the rating caps and match tables needed to reproduce the estimates.

Unanchored ratings The unanchored results show that the proposed latent score becomes more useful as search deepens. Depth gives the largest gains, while increasing width mainly helps when moving from $W \in \{1, 2\}$ to $W=3$; by $W=5$, additional expansion adds little strength. This plateau supports the intended contribution of representation-guided pruning: the best continuations are usually already among the top three candidates. At shallow depths, SOLIS Base leads Mini by roughly 100–250 Elo, while at depths 4 and 5 Mini becomes competitive with Base and even exceeds it for $W \in \{3, 5\}$ under this scoring rule.

Anchored ratings Anchored scoring gives a cleaner scaling pattern: Elo increases smoothly with depth and width, and Base remains about 100–250 Elo stronger than Mini. Both models exceed 2500 Elo at depth 5. Unlike the unanchored case, Base is consistently stronger across settings, and anchoring gives Base a steady 10–30 Elo improvement. This result makes the scoring contribution more precise: measuring projection relative to μ_{Black} appears to stabilize comparisons between candidate continuations.

4.2 EQUAL-DEPTH SIMULATIONS

The second experimental question is whether the same representation-guided search remains competitive when SOLIS and Stockfish are compared at fixed depths. Based on the Elo ablation in Section 4.1, we set the SOLIS branching factor to $W=3$, because wider searches gave only marginal gains and the top-three continuations captured most of the useful signal. Stockfish uses its default expansion with no fixed branching factor. For each depth $S \in \{3, 4, 5\}$, we simulate 200 alternating-color games and report both unanchored and anchored results in Table 2.

Simulation results These equal-depth matches show where the latent-planning contribution is strongest. With unanchored scoring, SOLIS Base scores more match points than Stockfish at every tested depth; Mini is competitive at depths 3 and 5 but trails at depth 4. With anchored scoring, Base narrowly loses at depth 3 but wins at depths 4 and 5, while Mini wins at depths 3 and 5 and again falls behind at depth 4.

Table 2: Equal-depth match points for SOLIS vs. Stockfish. The middle columns evaluate direct similarity to the global advantage vector, while the rightmost columns evaluate anchored projection. Higher scores indicate more match points; draws count as 0.5.

Config	Depth	Direct Similarity		Anchored Projection	
		SOLIS score	Stockfish score	SOLIS score	Stockfish score
Base	3	106	94	98	102
	4	109	91	108	92
	5	107.5	92.5	110	90
Mini	3	108	92	105	95
	4	85	115	91	109
	5	101	99	113	87

Nodes searched The computational result is that SOLIS obtains this competitiveness with a very small search tree. With branching factor $W=3$, the maximum number of SOLIS nodes at depth S is $N(S) = \sum_{i=0}^S 3^i$, which gives $N(3) = 40$, $N(4) = 121$, and $N(5) = 364$. Stockfish searches a variable number of nodes depending on pruning and heuristics: at depth 3 it averages 137 nodes (maximum 1625), at depth 4 it averages 214 nodes (maximum 4463), and at depth 5 it averages 338 nodes (maximum 10611). Thus the comparison supports the paper’s main empirical claim: **much of the strength normally gained from broad expansion can be recovered with higher-quality representations.**

4.3 INTERPRETABLE GAME TRAJECTORIES

The third result concerns interpretability, a direct benefit of making planning geometric. Figure 5 plots latent move sequences from three representative games: a White win, a balanced draw, and a Black win. Each position is embedded independently, projected onto the same 2D sample space, and connected with arrows to show temporal movement. Decisive games tend to move smoothly toward the corresponding winning region, while closely contested games remain near the center. These trajectories complement Figure 1 by showing that the learned advantage geometry is visible not only in static positions but also across actual games.

432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485

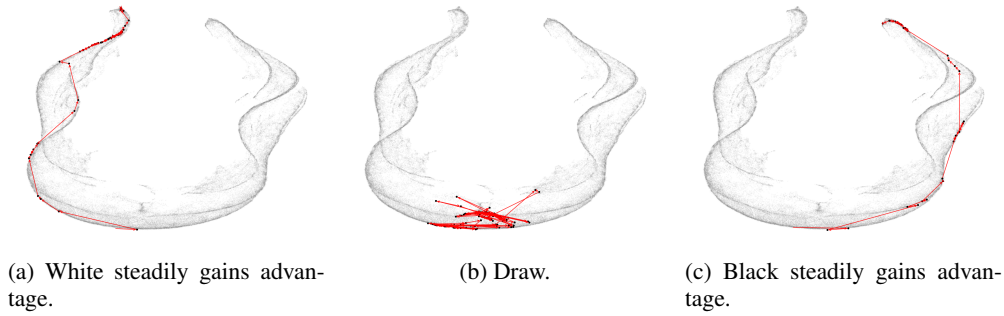


Figure 5: Game trajectories in the SOLIS representation. Three games are embedded in the same latent projection; red arrows trace position-by-position movement and show how wins and draws occupy different regions of the evaluation-aligned space.

5 DISCUSSION

5.1 LIMITATIONS

Blindness to threefold repetition A limitation of SOLIS follows directly from the same simplification that makes the contribution lightweight: actions are selected by alignment with an advantage vector, not by reasoning over complete game history. As a result, the engine can miss repetition-based drawing rules. In chess, if the same position, including side to move, castling rights, and en-passant rights, occurs three times, either player can claim a draw. A strategic opponent can therefore force repetitions, such as perpetual checks, to neutralize positions that SOLIS would otherwise convert. Game *C* in Appendix A.4 illustrates this failure mode: SOLIS reaches a winning position against Stockfish but permits a premature repetition draw.

Indecisiveness in winning positions A second limitation is that the evaluation-aligned embedding does not encode how quickly a forced win is achieved. For example, mate-in-1 and mate-in-20 both have win probability 1.0, even though the shorter line is strategically preferable. If SOLIS chooses a longer winning continuation, the opponent may gain opportunities to complicate the position, sometimes producing avoidable draws or losses. Ruoss et al. (2024) observed the same issue and avoided it with Stockfish as an external oracle. SOLIS instead follows its own latent score, which keeps the method self-contained but leaves this gap unresolved. Game *I* in Appendix A.5 shows a case where many continuations remain winning for several moves, yet suboptimal choices lead to a draw.

5.2 CONCLUSION

This paper contributes a concrete test of evaluation-aligned latent planning. We introduced SOLIS, which replaces policy/value heads and learned dynamics with a supervised contrastive embedding where proximity reflects outcome similarity. A single global *advantage vector* then turns that representation into a planner: candidate actions are ranked by alignment with the direction from losing to winning regions and expanded with a small search.

The chess experiments make the contribution measurable. With a fixed branching factor $W=3$ and shallow depths $S \in \{3, 4, 5\}$, SOLIS matches or exceeds a time-capped Stockfish in several equal-depth matchups while searching far fewer nodes per move: 40–364 for SOLIS versus hundreds to thousands for Stockfish at the same depths. Elo improves with depth; with anchored scoring, both Mini and Base exceed 2500 Elo at depth five, and anchoring gives Base a consistent 10–30 Elo gain. Width adds little beyond three candidates, supporting the claim that the representation makes strong continuations salient. The same space also supports qualitative inspection, with decisive games moving toward winning regions and draws remaining near neutral.

Overall, the paper’s supported conclusion is that evaluation-aligned geometry can serve as a practical planning substrate when reliable state evaluations are available. This does not replace all search

machinery, but it offers a compute-efficient alternative to broad rollout-based search or explicit dynamics learning in settings where low-branching action selection is valuable. Future work should address the limitations exposed here: adding rule-aware state features, preferring shorter forced wins, combining SOLIS with policy priors or MCTS, studying calibration and uncertainty, and testing transfer beyond chess.

REPRODUCIBILITY STATEMENT

To make the contribution reproducible, Appendix A.2 reports the full training setup, including hyperparameters and implementation details. Section 3 specifies the input representation, encoder architecture, supervised contrastive objective, and inference procedure, with model configurations in Table 1. Section 4 and Appendix A.3 describe the match setup, Stockfish configuration, and rating calculation. Appendix A.6 provides detailed match outcomes for verifying the Elo estimates.

REFERENCES

- T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations, 2020. URL <https://arxiv.org/abs/2002.05709>.
- S. Chowdhury, G. Hamerly, and M. McGarrity. Active learning strategy using contrastive learning and k-means for aquatic invasive species recognition. In *2024 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, pages 848–858, 2024. doi: 10.1109/WACVW60836.2024.00097.
- R. Coulom. Whole-history rating: A bayesian rating system for players of time-varying strength. In *Computers and Games*, volume 5131 of *LNC3*, pages 113–124. Springer, 2008. doi: 10.1007/978-3-540-87608-3_10.
- J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL <https://arxiv.org/abs/1810.04805>.
- A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. URL <https://arxiv.org/abs/2010.11929>.
- R. Gieslmann and F. T. Pokorny. Latent planning via expansive tree search. In A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=zSdz5scsnzU>.
- R. Girdhar, A. El-Nouby, Z. Liu, M. Singh, K. V. Alwala, A. Joulin, and I. Misra. Imagebind: One embedding space to bind them all, 2023. URL <https://arxiv.org/abs/2305.05665>.
- D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels, 2019. URL <https://arxiv.org/abs/1811.04551>.
- D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination, 2020. URL <https://arxiv.org/abs/1912.01603>.
- D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering atari with discrete world models, 2022. URL <https://arxiv.org/abs/2010.02193>.
- D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models, 2024. URL <https://arxiv.org/abs/2301.04104>.
- J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, T. Pfaff, T. Weber, L. Buesing, and P. W. Battaglia. Combining q-learning and search with amortized value estimates, 2020. URL <https://arxiv.org/abs/1912.02807>.
- K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition, 2015. URL <https://arxiv.org/abs/1512.03385>.

-
- D. Hendrycks and K. Gimpel. Gaussian error linear units (gelus), 2023. URL <https://arxiv.org/abs/1606.08415>.
- M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization, 2021. URL <https://arxiv.org/abs/1906.08253>.
- Z. Jiang, T. Zhang, M. Janner, Y. Li, T. Rocktäschel, E. Grefenstette, and Y. Tian. Efficient planning in a compact latent action space, 2023. URL <https://arxiv.org/abs/2208.10291>.
- P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan. Supervised contrastive learning, 2021. URL <https://arxiv.org/abs/2004.11362>.
- M. Lai. Giraffe: Using deep reinforcement learning to play chess, 2015. URL <https://arxiv.org/abs/1509.01549>.
- L. McInnes, J. Healy, and J. Melville. Umap: Uniform manifold approximation and projection for dimension reduction, 2020. URL <https://arxiv.org/abs/1802.03426>.
- T. M. Moerland, J. Broekens, A. Plaat, and C. M. Jonker. Model-based reinforcement learning: A survey, 2022. URL <https://arxiv.org/abs/2006.16712>.
- Y. Nasu. Efficiently updatable neural-network-based evaluation functions for computer shogi, 2018. Accessed: 2024-04-25.
- G.-C. Pascutto and G. Linscott. Leelachesszero, 2018. URL <https://lczero.org>. Accessed: 2024-04-25.
- A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019. URL <https://arxiv.org/abs/1912.01703>.
- M. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. Wiley, 2014. ISBN 9781118625873. URL <https://books.google.com/books?id=VvBjBAAQBAJ>.
- A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision, 2021. URL <https://arxiv.org/abs/2103.00020>.
- T. Romstad, M. Costalba, J. Kiiski, G. Linscott, Y. Nasu, M. Isozaki, and H. Noda. Stockfish, 2008. URL <https://stockfishchess.org>. Accessed: 2025-04-22.
- A. Ruoss, G. Delétang, S. Medapati, J. Grau-Moya, L. K. Wenliang, E. Catt, J. Reid, C. A. Lewis, J. Veness, and T. Genewein. Amortized planning with large-scale transformers: A case study on chess, 2024. URL <https://arxiv.org/abs/2402.04494>.
- S. Russell and P. Norvig. *Artificial Intelligence, A modern approach*. 1995.
- J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, Dec. 2020. ISSN 1476-4687. doi: 10.1038/s41586-020-03051-4. URL <http://dx.doi.org/10.1038/s41586-020-03051-4>.
- C. E. Shannon. XXII. programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950.
- D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017. URL <https://arxiv.org/abs/1712.01815>.

-
- N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15(1):1929–1958, Jan. 2014. ISSN 1532-4435.
- R. S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4):160–163, 1991.
- R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- J. v. Neumann. Zur theorie der gesellschaftsspiele. *Mathematische annalen*, 100(1):295–320, 1928.
- A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding, 2019. URL <https://arxiv.org/abs/1807.03748>.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- A. L. Zobrist. A new hashing method with application for game playing. *ICGA Journal*, 13(2): 69–73, 1990.

A APPENDIX

A.1 USE OF LARGE LANGUAGE MODELS (LLMs)

Large language models were used only for minor manuscript-support tasks, including formatting and figure/table placement. They were not used to design experiments, analyze results, or generate technical content. The methodological and experimental contributions reported in the paper were developed and verified by the authors.

A.2 TRAINING SETUP

Both SOLIS configurations, Mini and Base, are trained with the same contribution-defining objective: supervised contrastive learning on 5M ChessBench positions. Training uses cosine similarity over ℓ_2 -normalized embeddings and SGD with momentum on 6 48 GB L40S GPUs for 400k steps with batch size 128.

Among these settings, τ and δ most directly shape the learned geometry. The temperature $\tau = 0.07$ follows prior contrastive work (Radford et al., 2021; Girdhar et al., 2023): smaller values sharpen similarity scores, while larger values oversmooth gradients. We tested $\tau \in \{0.05, 0.07, 0.1\}$ and found that 0.07 converged most reliably across model sizes. The positive margin $\delta = 0.05$ controls which evaluations share a neighborhood, balancing excessive collapse when δ is large against excessive fragmentation when δ is small; it also reflects Stockfish evaluation noise, since sub-0.05 differences are rarely meaningful in practice.

A.3 STOCKFISH SETUP

All Stockfish comparisons use the official Linux release of Stockfish 16 (Romstad et al., 2008) with 32 MB hash size and default threading. Rating-capped experiments use `UCI_LimitStrength` and `UCI_Elo`; equal-depth experiments set the `depth` parameter. To report node counts, we set the engine information level to `INFO_ALL` and extract the `nodes` attribute for each move.

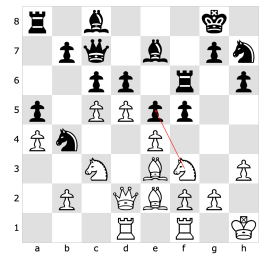
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701

Table 3: Training hyperparameters for the supervised contrastive objective that creates the SOLIS evaluation-aligned embedding space.

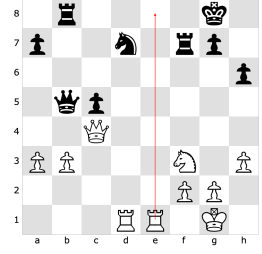
Attribute	Value
Loss	SupCon
τ	0.07
δ (positive margin)	0.05
Positives per anchor	5
Optimizer	SGD
Momentum	0.9
Learning rate	0.05
Batch size	128
Steps	400k
Dropout	0.10
Activation	GELU
Similarity	cosine
Embedding norm	ℓ_2
Token length	77

A.4 REPRESENTATIVE GAMES: STOCKFISH (CAP) VS. SOLIS (DEPTH, WIDTH)

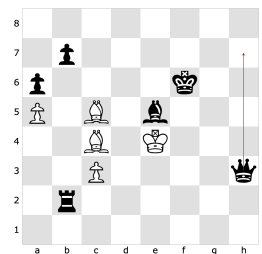
These representative games make the qualitative contribution easier to inspect. SOLIS is limited to search depth S and width W , while Stockfish uses unbounded search under a rating cap. Games *A*, *B*, and *E* highlight tactical and long-horizon behavior produced by the latent planner; Games *C* and *D* illustrate characteristic weaknesses. In each table, the left column gives the game in Portable Game Notation (PGN), and the right column shows a critical position.

1. d4 Nf6 2. c4 d6 3. Nc3 e5 4. Nf3 Qe7 5. e4 c6 6. d5 Qc7 7. Be2 h6 8. a4 a5 9. O-O Be7 10. h3 Na6 11. Be3 Nb4 12. Kh1 Nh7 13. Qd2 O-O 14. Rad1 f5 15. c5 Rf6 16. Nxe5 f4 17. Bxf4 Rxf4 18. Qxf4 dxe5 19. d6 exf4 20. Bc4+ Kf8 21. dxc7 Ke8 22. e5 Ng5 23. Rd6 Bxd6 24. exd6 Bd7 25. Re1+ Kf8 26. Re7 Bf5 27. h4 Nd5 28. Nxd5 cxd5 29. Bxd5 Be6 30. Bxe6 f3 31. hxg5 b6 32. Bh3 hxg5 33. Re5 Rc8 34. Rf5+ Kg8 35. Rf7 g4 36. cxb6 g6 37. d7 fxg2+ 38. Kh2 Rf8 39. Bxg2 Kxf7 40. Bc6 Ke7 41. Kg3 Rf3+ 42. Kh2 Rd3 43. b7 Rh3+ 44. Kg2 Re3 45. d8=Q+ Kf7 46. b8=Q Rg3+ 47. Kh2 Rh3+ 48. Kg1 Rh7 49. Qe8+ Kf6 50. Qbd8+ Kf5 51. Be4+ Kf4 52. Qf6# 1-0	
--	---

Game A — SOLIS ($S=5$, $W=3$) vs. SF 2350, SOLIS–White. Result: 1–0. The latent-guided search selects a knight sacrifice that creates a middlegame advantage.

1. d4 d5 2. e3 c6 3. Bd3 h6 4. h3 Nf6 5. Nf3 e6 6. O-O b6 7. c4 Ba6 8. b3 Nbd7 9. Bb2 Bb4 10. a3 Be7 11. Nbd2 O-O 12. Re1 c5 13. Qe2 Qc7 14. e4 Nxe4 15. Nxe4 dxe4 16. Bxe4 Rab8 17. d5 Bf6 18. dxe6 Bxb2 19. exf7+ Rxf7 20. Qxb2 Bb7 21. Bxb7 Qxb7 22. Rad1 b5 23. Qe2 bxc4 24. Qxc4 Qb5 25. Re8+ Nf8 26. Rxb8 Qxb8 27. Rd5 Qf4 28. Qxf4 Rxf4 29. Rxc5 Ne6 30. Rc6 Nd4 31. Nxd4 a5 32. Rc4 Rf8 33. Nc6 Kh8 34. Nxa5 Ra8 35. b4 Rd8 36. g3 Rd3 37. a4 g5 38. b5 Rd2 39. b6 Rd5 40. b7 Rxa5 41. Rc8+ Kh7 42. b8=Q Rd5 43. Rc7+ Kg6 44. Qg8+ Kf5 45. Qxd5+ Kg6 46. Qf7# 1-0	
--	--

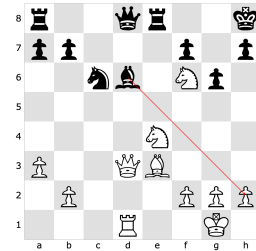
Game B — SOLIS ($S=5$, $W=3$) vs. SF 2300, SOLIS–White. Result: 1–0. SOLIS identifies a rook sacrifice that wins Stockfish’s queen and converts the resulting advantage.

1. e3 Nf6 2. Nc3 d5 3. Nf3 c5 4. d4 e6 5. g3 a6 6. a4 Nc6 7. Ne2 Bd6 8. Bg2 O-O 9. h3 Re8 10. c3 Qc7 11. Nd2 e5 12. Nb3 c4 13. dxe5 Nxe5 14. Nbd4 Nd3+ 15. Kf1 Nc5 16. Bf3 Bd7 17. Kg2 Be5 18. a5 Rad8 19. Qc2 g6 20. Bd2 h5 21. Be1 h4 22. gxh4 Kg7 23. Ng3 Rh8 24. h5 Rdg8 25. hxg6 fxg6 26. Be2 Qd6 27. Qd1 Qe7 28. f3 Bd6 29. Nf1 Rh5 30. h4 Rgh8 31. Bf2 R5h7 32. Qe1 Bb8 33. Bg3 Ba7 34. Bf2 Nh5 35. Ng3 Bb8 36. f4 Nf6 37. h5 Rh6 38. hxg6 Rxh1 39. Qxh1 Qe8 40. Ndf5+ Bxf5 41. Nxf5+ Kxg6 42. Ne7+ Kf7 43. e4 Rxh1 44. Rxh1 Qxe7 45. Bxc5 Qxe4+ 46. Bf3 Qg6+ 47. Kf1 Bxf4 48. b4 cxb3 49. Rh8 Ke6 50. Rf8 b2 51. Rxf6+ Kxf6 52. Bxd5 b1=R+ 53. Ke2 Rb2+ 54. Kf3 Be5 55. Bc4 Qh5+ 56. Ke3 Qh3+ 57. Ke4 Qh7+ 58. Kf3 Qh3+ 59. Ke4 Qh7+ 60. Kf3 Qh3+ 61. Ke4 Qh7+ 62. Kf3 Qh3+ 63. Ke4 Qh7+ 64. Kf3 Qh3+ 65. Ke4 1/2–1/2	
--	---

Game C — SF 2200 vs. SOLIS ($S=5$, $W=3$), SOLIS–Black. Result: 1/2–1/2. This game highlights the repetition limitation: SOLIS reaches a winning endgame but accepts a draw because move history is not part of its latent score.

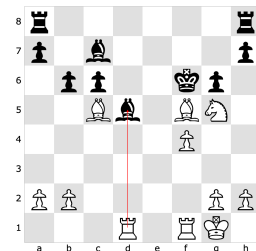
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809

1. d4 d5 2. c4 dxc4 3. e3 e5 4. Bxc4 exd4 5. exd4
Nf6 6. Nc3 Bd6 7. Qe2+ Qe7 8. Be3 O-O 9. Nf3 c6 10.
O-O Be6 11. Rfd1 Re8 12. d5 cxd5 13. Bxd5 Nc6 14.
a3 Nxd5 15. Nxd5 Qd8 16. Qd3 Bg4 17. Ng5 g6 18. Ne4
Bxd1 19. Ndf6+ Kh8 20. Rxd1 Bxh2+ 21. Kxh2 Qxd3 22.
Rxd3 Red8 23. Nd7 Kg7 24. f3 h6 25. Rd6 f5 26. Nec5
Re8 27. Bd2 Re2 28. Kh1 Kf7 29. b3 Ke7 30. Nxb7 Rc8
31. Ndc5 Kf7 32. Kg1 Kg8 33. Rxg6+ Kf7 34. Rd6 Ne7
35. Rd7 Rg8 36. Nd6+ Kg6 37. Nc4 Kh5 38. Ne3 f4 39.
Ne6 fxe3 40. Nf4+ Kg5 41. Nxe2 exd2 42. Kf2 Kf6 43.
Nc3 Ke6 44. Rd4 Rc8 45. Ne4 d1=N+ 46. Rxd1 Rc2+ 47.
Kg1 Nf5 48. Rd2 Rxd2 49. Nxd2 Ke5 50. Kh2 Kf4 51.
b4 Ke3 52. Ne4 Nd4 53. a4 Nc2 54. b5 Nd4 55. Nd6
Ne6 56. a5 Kd3 57. g4 Kd4 58. b6 axb6 59. axb6 Nc5
60. Kg3 Kd5 61. b7 Na6 62. Nf7 Kc6 63. Nxh6 Kxb7
64. f4 Kc7 65. Kh4 Kd7 66. Kh5 Nc5 67. f5 Kd6 68.
f6 Ne6 69. Nf7+ Kc6 70. Kh6 Kd5 71. g5 Nc5 72. Kg6
Ne6 73. Kh5 Kd4 74. g6 Kc4 75. Ne5+ Kd5 76. Ng4 Ke4
77. Kh6 Kf5 78. g7 Nxg7 79. Kxg7 Kxg4 80. Kf7 Kf5
81. Ke7 Kg5 82. f7 Kf4 83. Kf6 Kf3 84. Ke5 Kg3 85.
Ke4 Kf2 86. Kd3 Kg1 87. Ke3 Kh2 88. f8=Q Kg3 89.
Qg8+ Kh3 90. Kf4 Kh2 91. Kf3 Kh3 92. Qg3# 1-0



Game D — SF 2500 vs. SOLIS ($S=5, W=3$), SOLIS–Black. Result: 1-0 This failure case shows the latent score overvaluing a short-term sacrifice that becomes a long-term material disadvantage.

1. d4 d5 2. e3 Nd7 3. c4 e5 4. dxe5 dxc4 5. Nf3
Qe7 6. Bxc4 Nxe5 7. Nxe5 Qxe5 8. Qb3 Qh5 9. e4
Nf6 10. O-O Bd6 11. Qb5+ c6 12. Qxh5 Nxh5 13. Nc3
Be5 14. f4 Bc7 15. Be3 g6 16. Bb3 Be6 17. Rad1 Ng7
18. Bc2 f5 19. Bc5 Kf7 20. exf5 Nxf5 21. Ne4 b6
22. Ng5+ Kf6 23. Bxf5 Bd5 24. Rxd5 cxd5 25. Bd4+
Kxf5 26. h3 Bxf4 27. Bxh8 Kxg5 28. Bg7 Rg8 29. Bd4
Re8 30. Rf3 Re1+ 31. Kf2 Re6 32. g3 Bd2 33. Rf7
Re4 34. Bf6+ Kh6 35. g4 a6 36. Kg3 d4 37. h4 Be1+
38. Kh3 Re3+ 39. Kg2 Bxh4 40. Bxh4 a5 41. Bf6 a4
42. Rd7 Rd3 43. Rd5 b5 44. Rxb5 a3 45. Re5 Rd2+
46. Kf3 axb2 47. g5+ Kh5 48. Rb5 Rd3+ 49. Kf4 h6
50. gxh6+ Kxh6 51. Rxb2 g5+ 52. Kg4 Rd1 53. Bxg5+
Kg6 54. Rb6+ Kg7 55. Kf5 Kf7 56. Rb7+ Ke8 57. a4
Ra1 58. Re7+ Kf8 59. Rh7 Rxa4 60. Bh6+ Kg8 61. Re7
Rc4 62. Kg5 Rc5+ 63. Kg4 Ra5 64. Rg7+ Kh8 65. Re7
d3 66. Rd7 d2 67. Bxd2 Rc5 68. Bf4 Kg8 69. Be3 Rb5
70. Bh6 Re5 71. Kf4 Rc5 72. Ke4 Rb5 73. Kf4 Rb4+
74. Ke5 Rb1 75. Kf6 Rb2 76. Kg6 Rb6+ 77. Kh5 Rb2
78. Bg5 Rb6 79. Bh6 Re6 80. Kg5 Ra6 81. Rg7+ Kh8
82. Re7 Ra5+ 83. Kg6 Ra6+ 84. Kh5 Kg8 85. Rb7 Rc6
86. Rd7 Rc5+ 87. Bg5 Rc2 88. Kg6 Rc6+ 89. Bf6 Re6
90. Rd8+ Re8 91. Rxe8# 1-0

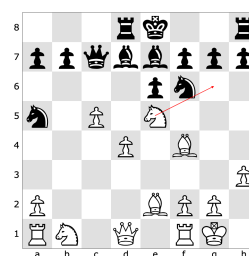


Game E — SOLIS ($S=5, W=3$) vs. Stockfish 2500, SOLIS–White. Result: 1-0 SOLIS chooses an exchange sacrifice for long-term compensation and later converts the advantage.

A.5 EQUAL-DEPTH MATCHES: SOLIS D vs. STOCKFISH D

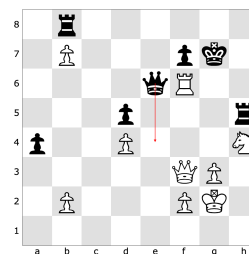
We also include illustrative games in which SOLIS and Stockfish search to the same depth D . These examples support the equal-depth contribution by showing how latent planning competes head-to-head without a Stockfish rating cap. As in the previous appendix tables, the left column gives the game in Portable Game Notation (PGN), and the right column shows a critical position.

1. d4 d5 2. e3 c5 3. Nf3 Nf6 4. Be2 e6 5. O-O Nc6
6. b3 cxd4 7. exd4 Be7 8. c3 Qc7 9. h3 Bd7 10. c4
dxc4 11. bxc4 Na5 12. Ne5 Rd8 13. Bf4 Bd6 14. c5
Be7 15. Ng6 Qc8 16. Nxh8 Bc6 17. Nc3 Kf8 18. Rc1
Qa8 19. Bc7 b5 20. Bxa5 Bxg2 21. d5 Bxh3 22. Qd3
Bf5 23. Qg3 b4 24. Rfd1 Bxc5 25. Qc7 Be7 26. Nb5
Nxd5 27. Rxd5 Qxd5 28. Bxb4 Qd7 29. Qxd7 Re8 30.
Rc8 g5 31. Qxe7+ Kg7 32. Qxf7+ Kh6 33. Bf8+ Rxf8 34.
Qxf8# 1-0



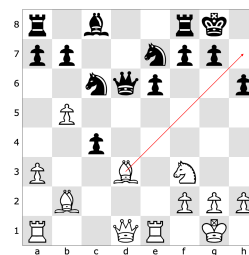
Game F — SOLIS $S = 5$ vs. Stockfish $S = 5$, SOLIS-White. Result: 1-0 SOLIS uses a middlegame knight sacrifice to obtain a concrete tactical advantage.

1. d4 d5 2. e3 c5 3. Nf3 e6 4. Be2 cxd4 5. exd4
Bd6 6. O-O Nf6 7. Bg5 Nbd7 8. Nbd2 h6 9. Bh4 O-O
10. Re1 a5 11. Bf1 Bf4 12. c3 Qb6 13. Qc2 Qc7 14.
a4 b6 15. Bb5 Ba6 16. Bxa6 Rxa6 17. Qd3 Ra7 18. Re2
Qc8 19. Reel Nh5 20. Nf1 Nh6 21. N1d2 Nh5 22. Nf1
Bc7 23. Qd1 Nh6 24. Qd3 Nh5 25. Qd1 Nh6 26. Qd3
Rb7 27. Qb5 Ra7 28. N1d2 Nh5 29. h3 g5 30. Rac1
gxh4 31. c4 Nf4 32. Rc3 Qd8 33. g3 Nxh3+ 34. Kg2
Ng5 35. cxd5 h3+ 36. Kg1 h2+ 37. Kxh2 exd5 38. Rc6
Nf6 39. Nxg5 hxg5 40. Kg2 Re8 41. Rh1 Bd6 42. Nf3
Kg7 43. Nxg5 Rc7 44. Rxd6 Qxd6 45. Qf1 b5 46. axb5
Rcc8 47. Rh4 Rh8 48. Nf3 Ne4 49. Rf4 Rh5 50. Qe2
Qe6 51. Nh4 Rch8 52. b6 Nf6 53. Qf3 a4 54. b7 Rb8
55. Rxf6 Qe4 56. Rxf7+ Kh8 57. Qxe4 dxe4 58. Rc7
Rg5 59. Rc8+ Rg8 60. Rc7 Rg5 61. Rc8+ Rg8 62. Ng6+
Kg7 63. Ne7 Rf8 64. Rc7 Rf7 65. d5 Rbf8 66. Nf5+
Kg6 67. Rxf7 Kxf7 68. Nd6+ Kg6 69. Nc8 Kg5 70. b8=Q
Kg4 71. Qc7 Rf5 72. Nd6 e3 73. Qg7+ Rg5 74. f3+ Kh5
75. Qh7# 1-0



Game G — SOLIS $S = 5$ vs. Stockfish $S = 5$, SOLIS-White. Result: 1-0 After Stockfish builds an advantage and then blunders in the late middlegame, SOLIS recognizes the changed evaluation path and converts in the endgame.

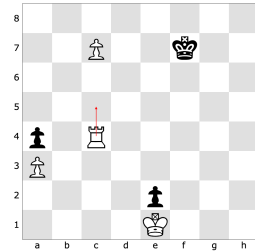
1. d4 c6 2. e4 d5 3. Nc3 Nf6 4. e5 Ng8 5. Bd3 e6
6. Nf3 c5 7. dxc5 Bxc5 8. O-O Nc6 9. a3 h6 10. b4
Bb6 11. Nb5 Bc7 12. Re1 Bb8 13. c4 Nge7 14. Nd6+
Bxd6 15. exd6 Qxd6 16. Bb2 O-O 17. b5 dxc4 18. Bh7+
Kh8 19. Qxd6 f6 20. Be4 c3 21. Bxc3 e5 22. bxc6 Bf5
23. Qxe7 Bxe4 24. Rxe4 bxc6 25. Nxe5 Rfe8 26. Nf7+
Kg8 27. Nxh6+ Kh7 28. Rae1 Rad8 29. Ba5 Rd4 30. g4
Rxe4 31. Rxe4 Rb8 32. Qxa7 Rd8 33. Rf4 Rd3 34. g5
fxg5 35. Rd4 Rxa3 36. Kf1 Rb3 37. Rb4 Rc3 38. h4
gxh4 39. Rd4 h3 40. Qe7 h2 41. Qe4+ Kxh6 42. Rd6+
Kg5 43. Rg6+ Kh5 44. Qg4# 1-0



Game H — SOLIS $S = 3$ vs. Stockfish $S = 3$, SOLIS-White. Result: 1-0 SOLIS finds a bishop sacrifice that enables a discovered attack on Stockfish's queen, then converts the material advantage.

864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917

1. d4 d5 2. e3 c6 3. c4 e6 4. Nf3 Nf6 5. Be2 c5
6. O-O Nc6 7. b3 a6 8. Bb2 Be7 9. a3 cxd4 10. exd4
O-O 11. c5 b6 12. b4 bxc5 13. dxc5 Bb7 14. Nbd2 a5
15. b5 Bxc5 16. Rcl Qb6 17. bxc6 Ng4 18. Rxc5 Ba6
19. Rc2 Nxf2 20. Rxf2 Bxe2 21. Qxe2 a4 22. c7 Rac8
23. Bd4 Qb5 24. Qxb5 Rfe8 25. Qb8 h6 26. Ne5 g5 27.
h4 Rf8 28. Ne4 dxe4 29. Rxf7 Rxb8 30. Rxf8+ Rxf8 31.
hxg5 Rc8 32. gxh6 e3 33. Bxe3 Kh7 34. Rc4 Kg8 35.
Kh2 Kh7 36. g4 Rg8 37. g5 Ra8 38. Rg4 Kh8 39. Ba7
Rf8 40. Bb8 Rf2+ 41. Kg1 Rc2 42. Rc4 Rb2 43. Ba7
Rb1+ 44. Kg2 Kh7 45. Bf2 Rb2 46. g6+ Kxh6 47. Kh2
Rxf2+ 48. Kg3 Rg2+ 49. Kxg2 Kg7 50. Rc1 Kf6 51. Kh2
Kg7 52. Nd7 e5 53. Nf6 Kxf6 54. Rg1 Kg7 55. Rf1 Kh6
56. Rg1 Kg7 57. Rf1 Kxg6 58. Rc1 Kg7 59. Rc4 Kf7
60. Kg2 Ke7 61. Kf2 Kd6 62. Ke1 Kd5 63. Rc1 Ke4 64.
Ke2 Kd4 65. Rc2 e4 66. Rc1 Kd5 67. Ke1 Ke5 68. Rc4
Kd5 69. Rc2 Kd6 70. Rc1 Ke7 71. Rc4 Kf7 72. Rc5 e3
73. Kf1 Ke7 74. Rc4 Kf7 75. Kg2 e2 76. Kf2 Ke7 77.
Ke1 Kf7 78. Rc5 Kg7 79. Rc2 Kf6 80. Rc4 Kf7 81. Rc5
Kg6 82. Kd2 Kf7 83. Ke1 Ke7 84. Rc4 Kf7 85. Rc5 Kg6
86. Rc4 Kf7 87. Rc5 1/2-1/2



Game I — SOLIS $S = 3$ vs. Stockfish $S = 3$, SOLIS–White. Result: 1/2-1/2 This draw illustrates an unresolved limitation: when many continuations remain winning, SOLIS does not reliably prefer the shortest forcing path.

A.6 SOLIS VS. STOCKFISH RESULTS

For reproducibility of the strength claims, we report the full SOLIS–Stockfish match outcomes for SOLIS search depths $S \in \{2, 3, 4, 5\}$. Each table indexes Stockfish Elo caps by row and SOLIS search width W by column. Entries list win–loss–draw counts, with total match points in parentheses. We provide separate tables for Base and Mini, and for both unanchored and anchored scoring, so the contribution of each planning rule can be verified directly.

Base: Depth 2					Base: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	32–46–22 (55)	41–47–12 (64)	–	–	1500	–	–	–	–
1550	39–38–23 (58)	37–50–13 (62)	–	–	1550	–	–	–	–
1600	32–44–24 (54)	33–51–16 (58)	–	–	1600	–	–	–	–
1650	33–41–26 (54)	29–46–25 (52)	–	–	1650	–	–	–	–
1700	34–38–28 (53)	22–53–25 (48)	–	–	1700	–	–	–	–
1750	37–35–28 (54)	30–49–21 (54)	–	–	1750	–	–	–	–
1800	29–42–29 (50)	36–42–22 (57)	–	–	1800	39–29–32 (54)	37–49–14 (62)	–	–
1850	–	–	–	–	1850	35–32–33 (51)	30–50–20 (55)	–	–
1900	–	–	–	–	1900	37–30–33 (52)	40–41–19 (60)	–	–
1950	–	–	–	–	1950	31–28–41 (45)	37–44–19 (59)	–	–
2000	–	–	34–40–26 (54)	43–31–26 (58)	2000	41–27–32 (54)	33–45–22 (56)	–	–
2050	–	–	46–38–16 (65)	36–38–26 (55)	2050	39–27–34 (52)	31–41–28 (52)	–	–
2100	–	–	34–45–21 (56)	34–31–35 (50)	2100	41–27–32 (54)	36–38–26 (55)	–	–
2150	–	–	46–31–23 (62)	30–43–27 (52)	2150	–	–	–	–
2200	–	–	31–35–34 (48)	35–28–37 (49)	2200	–	–	33–42–25 (54)	30–41–29 (50)
2250	–	–	34–34–32 (51)	40–29–31 (54)	2250	–	–	35–41–24 (56)	33–43–24 (54)
2300	–	–	32–32–36 (48)	37–34–29 (54)	2300	–	–	35–33–32 (52)	36–34–30 (53)
2350	–	–	–	–	2350	–	–	37–37–26 (56)	29–37–34 (48)
2400	–	–	–	–	2400	–	–	34–36–30 (52)	37–38–25 (56)
2450	–	–	–	–	2450	–	–	37–28–35 (51)	43–29–28 (58)
2500	–	–	–	–	2500	–	–	41–30–29 (56)	34–30–36 (49)
2550	–	–	–	–	2550	–	–	–	–
2600	–	–	–	–	2600	–	–	–	–
2650	–	–	–	–	2650	–	–	–	–
2700	–	–	–	–	2700	–	–	–	–

Base: Depth 4					Base: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	–	–	–	–	1500	–	–	–	–
1550	–	–	–	–	1550	–	–	–	–
1600	–	–	–	–	1600	–	–	–	–
1650	–	–	–	–	1650	–	–	–	–
1700	–	–	–	–	1700	–	–	–	–
1750	–	–	–	–	1750	–	–	–	–
1800	–	–	–	–	1800	–	–	–	–
1850	–	–	–	–	1850	–	–	–	–
1900	–	–	–	–	1900	–	–	–	–
1950	37–26–37 (50)	41–42–17 (62)	–	–	1950	36–34–30 (53)	38–41–21 (58)	–	–
2000	43–20–37 (53)	31–43–26 (52)	–	–	2000	31–28–41 (45)	27–52–21 (53)	–	–
2050	38–27–35 (52)	40–38–22 (59)	–	–	2050	41–24–35 (53)	36–37–27 (54)	–	–
2100	42–25–33 (54)	32–48–20 (56)	–	–	2100	43–23–34 (54)	36–39–25 (56)	–	–
2150	36–23–41 (48)	31–47–22 (54)	–	–	2150	42–23–35 (54)	27–45–28 (50)	–	–
2200	46–13–41 (52)	39–39–22 (58)	–	–	2200	42–21–37 (52)	23–51–26 (48)	–	–
2250	–	–	–	–	2250	–	–	–	–
2300	–	–	–	–	2300	–	–	–	–
2350	–	–	–	–	2350	–	–	–	–
2400	–	–	–	–	2400	–	–	–	–
2450	–	–	40–39–21 (60)	39–38–23 (58)	2450	–	–	25–46–29 (48)	43–30–27 (58)
2500	–	–	38–38–24 (57)	41–40–19 (61)	2500	–	–	40–36–24 (58)	34–44–22 (56)
2550	–	–	44–20–36 (54)	42–29–29 (56)	2550	–	–	40–31–29 (56)	38–38–24 (57)
2600	–	–	46–17–37 (54)	37–23–40 (48)	2600	–	–	46–25–29 (58)	33–37–30 (52)
2650	–	–	41–18–41 (50)	44–22–34 (55)	2650	–	–	47–17–36 (56)	30–35–35 (48)
2700	–	–	45–16–39 (53)	42–22–36 (53)	2700	–	–	38–25–37 (50)	45–21–34 (56)

Figure 6: Full Elo-supporting match outcomes for SOLIS Base with unanchored advantage-vector planning against Stockfish. Each per-depth block reports W–D–L counts with match score in parentheses out of 100.

972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025

Mini: Depth 2					Mini: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	32–32–36 (48)	27–45–28 (50)	–	–	1350	–	–	–	–
1400	30–41–29 (50)	33–27–40 (46)	–	–	1400	–	–	–	–
1450	29–44–27 (51)	33–40–27 (53)	–	–	1450	–	–	–	–
1500	31–38–31 (50)	31–39–30 (50)	–	–	1500	29–38–33 (48)	28–43–29 (50)	–	–
1550	38–35–27 (56)	23–50–27 (48)	–	–	1550	41–36–23 (59)	36–37–27 (54)	–	–
1600	34–30–36 (49)	29–37–34 (48)	–	–	1600	37–30–33 (52)	37–30–33 (52)	–	–
1650	24–45–31 (46)	28–36–36 (46)	–	–	1650	32–38–30 (51)	30–40–30 (50)	–	–
1700	–	–	–	–	1700	28–37–35 (46)	21–43–36 (42)	–	–
1750	–	–	–	–	1750	36–33–31 (52)	28–40–32 (48)	–	–
1800	–	–	37–31–32 (52)	37–36–27 (55)	1800	39–24–37 (51)	29–35–36 (46)	–	–
1850	–	–	34–24–42 (46)	34–31–35 (50)	1850	–	–	–	–
1900	–	–	40–28–32 (54)	40–29–31 (54)	1900	–	–	–	–
1950	–	–	39–29–32 (54)	39–29–32 (54)	1950	–	–	–	–
2000	–	–	38–34–28 (55)	42–28–30 (56)	2000	–	–	34–35–31 (52)	38–21–41 (48)
2050	–	–	41–27–32 (54)	40–26–34 (53)	2050	–	–	38–34–28 (55)	34–33–33 (50)
2100	–	–	35–30–35 (50)	36–26–38 (49)	2100	–	–	33–34–33 (50)	34–29–37 (48)
2150	–	–	–	–	2150	–	–	31–35–34 (48)	38–25–37 (50)
2200	–	–	–	–	2200	–	–	38–18–44 (47)	31–26–43 (44)
2250	–	–	–	–	2250	–	–	36–23–41 (48)	45–26–29 (58)
2300	–	–	–	–	2300	–	–	40–22–38 (51)	34–26–40 (47)
2350	–	–	–	–	2350	–	–	–	–
2400	–	–	–	–	2400	–	–	–	–
2500	–	–	–	–	2500	–	–	–	–
Mini: Depth 4					Mini: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	–	–	–	–	1350	–	–	–	–
1400	–	–	–	–	1400	–	–	–	–
1450	–	–	–	–	1450	–	–	–	–
1500	–	–	–	–	1500	–	–	–	–
1550	–	–	–	–	1550	–	–	–	–
1600	36–42–22 (57)	35–31–34 (50)	–	–	1600	–	–	–	–
1650	29–40–31 (49)	36–33–31 (52)	–	–	1650	–	–	–	–
1700	33–34–33 (50)	36–32–32 (52)	–	–	1700	39–30–31 (54)	34–34–32 (51)	–	–
1750	43–26–31 (56)	28–38–34 (47)	–	–	1750	37–29–34 (52)	32–30–38 (47)	–	–
1800	38–24–38 (50)	30–37–33 (48)	–	–	1800	42–20–38 (52)	33–31–36 (48)	–	–
1850	49–23–28 (60)	29–33–38 (46)	–	–	1850	36–24–40 (48)	43–28–29 (57)	–	–
1900	–	–	–	–	1900	45–26–29 (58)	46–31–23 (62)	–	–
1950	–	–	–	–	1950	40–22–38 (51)	37–35–28 (54)	–	–
2000	–	–	–	–	2000	–	–	–	–
2050	–	–	–	–	2050	–	–	–	–
2100	–	–	–	–	2100	–	–	–	–
2150	–	–	31–31–38 (46)	44–21–35 (54)	2150	–	–	–	–
2200	–	–	35–29–36 (50)	48–27–25 (62)	2200	–	–	27–37–36 (46)	33–38–29 (52)
2250	–	–	37–31–32 (52)	37–28–35 (51)	2250	–	–	30–33–37 (46)	38–31–31 (54)
2300	–	–	27–37–36 (46)	36–34–30 (53)	2300	–	–	38–37–25 (56)	33–39–28 (52)
2350	–	–	22–42–36 (43)	37–35–28 (54)	2350	–	–	32–29–39 (46)	29–36–35 (47)
2400	–	–	36–29–35 (50)	37–26–37 (50)	2400	–	–	30–26–44 (43)	30–32–38 (46)
2500	–	–	–	–	2500	–	–	26–34–40 (43)	43–25–32 (56)

Figure 7: Full Elo-supporting match outcomes for SOLIS Mini with unanchored advantage-vector planning against Stockfish. Each per-depth block reports W–D–L counts with match score in parentheses out of 100.

1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079

Base: Depth 2					Base: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	32-46-22 (55)	41-47-12 (64)	-	-	1500	-	-	-	-
1550	39-38-23 (58)	37-50-13 (62)	-	-	1550	-	-	-	-
1600	32-44-24 (54)	33-51-16 (58)	-	-	1600	-	-	-	-
1650	33-41-26 (54)	29-46-25 (52)	-	-	1650	-	-	-	-
1700	34-38-28 (53)	22-53-25 (48)	-	-	1700	-	-	-	-
1750	37-35-28 (54)	30-49-21 (54)	-	-	1750	-	-	-	-
1800	29-42-29 (50)	36-42-22 (57)	-	-	1800	39-29-32 (54)	37-49-14 (62)	-	-
1850	-	-	-	-	1850	35-32-33 (51)	30-50-20 (55)	-	-
1900	-	-	-	-	1900	37-30-33 (52)	40-41-19 (60)	-	-
1950	-	-	-	-	1950	31-28-41 (45)	37-44-19 (59)	-	-
2000	-	-	34-40-26 (54)	43-31-26 (58)	2000	41-27-32 (54)	33-45-22 (56)	-	-
2050	-	-	46-38-16 (65)	36-38-26 (55)	2050	39-27-34 (52)	31-41-28 (52)	-	-
2100	-	-	34-45-21 (56)	34-31-35 (50)	2100	41-27-32 (54)	36-38-26 (55)	-	-
2150	-	-	46-31-23 (62)	30-43-27 (52)	2150	-	-	-	-
2200	-	-	31-35-34 (48)	35-28-37 (49)	2200	-	-	33-42-25 (54)	30-41-29 (50)
2250	-	-	34-34-32 (51)	40-29-31 (54)	2250	-	-	35-41-24 (56)	33-43-24 (54)
2300	-	-	32-32-36 (48)	37-34-29 (54)	2300	-	-	35-33-32 (52)	36-34-30 (53)
2350	-	-	-	-	2350	-	-	37-37-26 (56)	29-37-34 (48)
2400	-	-	-	-	2400	-	-	34-36-30 (52)	37-38-25 (56)
2450	-	-	-	-	2450	-	-	37-28-35 (51)	43-29-28 (58)
2500	-	-	-	-	2500	-	-	41-30-29 (56)	34-30-36 (49)
2550	-	-	-	-	2550	-	-	-	-
2600	-	-	-	-	2600	-	-	-	-
2650	-	-	-	-	2650	-	-	-	-
2700	-	-	-	-	2700	-	-	-	-
Base: Depth 4					Base: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	-	-	-	-	1500	-	-	-	-
1550	-	-	-	-	1550	-	-	-	-
1600	-	-	-	-	1600	-	-	-	-
1650	-	-	-	-	1650	-	-	-	-
1700	-	-	-	-	1700	-	-	-	-
1750	-	-	-	-	1750	-	-	-	-
1800	-	-	-	-	1800	-	-	-	-
1850	-	-	-	-	1850	-	-	-	-
1900	-	-	-	-	1900	-	-	-	-
1950	37-26-37 (50)	41-42-17 (62)	-	-	1950	36-34-30 (53)	38-41-21 (58)	-	-
2000	43-20-37 (53)	31-43-26 (52)	-	-	2000	31-28-41 (45)	27-52-21 (53)	-	-
2050	38-27-35 (52)	40-38-22 (59)	-	-	2050	41-24-35 (53)	36-37-27 (54)	-	-
2100	42-25-33 (54)	32-48-20 (56)	-	-	2100	43-23-34 (54)	36-39-25 (56)	-	-
2150	36-23-41 (48)	31-47-22 (54)	-	-	2150	42-23-35 (54)	27-45-28 (50)	-	-
2200	46-13-41 (52)	39-39-22 (58)	-	-	2200	42-21-37 (52)	23-51-26 (48)	-	-
2250	-	-	-	-	2250	-	-	-	-
2300	-	-	-	-	2300	-	-	-	-
2350	-	-	-	-	2350	-	-	-	-
2400	-	-	-	-	2400	-	-	-	-
2450	-	-	40-39-21 (60)	39-38-23 (58)	2450	-	-	25-46-29 (48)	43-30-27 (58)
2500	-	-	38-38-24 (57)	41-40-19 (61)	2500	-	-	40-36-24 (58)	34-44-22 (56)
2550	-	-	44-20-36 (54)	42-29-29 (56)	2550	-	-	40-31-29 (56)	38-38-24 (57)
2600	-	-	46-17-37 (54)	37-23-40 (48)	2600	-	-	46-25-29 (58)	33-37-30 (52)
2650	-	-	41-18-41 (50)	44-22-34 (55)	2650	-	-	47-17-36 (56)	30-35-35 (48)
2700	-	-	45-16-39 (53)	42-22-36 (53)	2700	-	-	38-25-37 (50)	45-21-34 (56)

Figure 8: Full Elo-supporting match outcomes for SOLIS Base with anchored advantage-vector planning against Stockfish. Each per-depth block reports W-D-L counts with match score in parentheses out of 100.

1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133

Mini: Depth 2					Mini: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	36-32-32 (52)	39-30-31 (54)	-	-	1350	-	-	-	-
1400	23-49-28 (48)	30-32-38 (46)	-	-	1400	-	-	-	-
1450	33-37-30 (52)	24-47-29 (48)	-	-	1450	-	-	-	-
1500	34-38-28 (53)	25-40-35 (45)	-	-	1500	29-40-31 (49)	35-33-32 (52)	-	-
1550	28-38-34 (47)	40-39-21 (60)	-	-	1550	34-35-31 (52)	37-29-34 (52)	-	-
1600	23-47-30 (46)	28-42-30 (49)	-	-	1600	32-36-32 (50)	32-39-29 (52)	-	-
1650	35-29-36 (50)	38-37-25 (56)	-	-	1650	34-30-36 (49)	37-33-30 (54)	-	-
1700	-	-	-	-	1700	36-30-34 (51)	34-35-31 (52)	-	-
1750	-	-	-	-	1750	34-25-41 (46)	34-30-36 (49)	-	-
1800	-	-	35-37-28 (54)	40-28-32 (54)	1800	32-29-39 (46)	32-31-37 (48)	-	-
1850	-	-	32-32-36 (48)	44-29-27 (58)	1850	-	-	-	-
1900	-	-	37-30-33 (52)	45-21-34 (56)	1900	-	-	-	-
1950	-	-	43-30-27 (58)	36-29-35 (50)	1950	-	-	-	-
2000	-	-	35-31-34 (50)	43-27-30 (56)	2000	-	-	46-32-22 (62)	38-27-35 (52)
2050	-	-	37-29-34 (52)	45-27-28 (58)	2050	-	-	37-27-36 (50)	39-26-35 (52)
2100	-	-	36-27-37 (50)	42-34-24 (59)	2100	-	-	29-34-37 (46)	48-17-35 (56)
2150	-	-	-	-	2150	-	-	37-33-30 (54)	39-25-36 (52)
2200	-	-	-	-	2200	-	-	37-31-32 (52)	41-23-36 (52)
2250	-	-	-	-	2250	-	-	36-23-41 (48)	44-20-36 (54)
2300	-	-	-	-	2300	-	-	30-43-27 (52)	38-26-36 (51)
2350	-	-	-	-	2350	-	-	-	-
2400	-	-	-	-	2400	-	-	-	-
2500	-	-	-	-	2500	-	-	-	-
Mini: Depth 4					Mini: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	-	-	-	-	1350	-	-	-	-
1400	-	-	-	-	1400	-	-	-	-
1450	-	-	-	-	1450	-	-	-	-
1500	-	-	-	-	1500	-	-	-	-
1550	-	-	-	-	1550	-	-	-	-
1600	26-45-29 (48)	31-33-36 (48)	-	-	1600	-	-	-	-
1650	37-39-24 (56)	28-32-40 (44)	-	-	1650	-	-	-	-
1700	23-39-38 (42)	29-41-30 (50)	-	-	1700	37-36-27 (55)	34-29-37 (48)	-	-
1750	36-29-35 (50)	38-30-32 (53)	-	-	1750	42-23-35 (54)	37-33-30 (54)	-	-
1800	35-24-41 (47)	33-36-31 (51)	-	-	1800	36-28-36 (50)	31-35-34 (48)	-	-
1850	41-21-38 (52)	30-28-42 (44)	-	-	1850	38-21-41 (48)	33-32-35 (49)	-	-
1900	-	-	-	-	1900	37-27-36 (50)	37-37-26 (56)	-	-
1950	-	-	-	-	1950	43-18-39 (52)	38-24-38 (50)	-	-
2000	-	-	-	-	2000	-	-	-	-
2050	-	-	-	-	2050	-	-	-	-
2100	-	-	-	-	2100	-	-	-	-
2150	-	-	39-23-38 (50)	32-30-38 (47)	2150	-	-	-	-
2200	-	-	33-40-27 (53)	38-25-37 (50)	2200	-	-	30-40-30 (50)	32-34-34 (49)
2250	-	-	36-31-33 (52)	30-27-43 (44)	2250	-	-	29-43-28 (50)	49-26-25 (62)
2300	-	-	32-33-35 (48)	37-28-35 (51)	2300	-	-	31-30-39 (46)	30-35-35 (48)
2350	-	-	35-34-31 (52)	40-25-35 (52)	2350	-	-	26-32-42 (42)	40-34-26 (57)
2400	-	-	28-26-46 (41)	26-38-36 (45)	2400	-	-	39-32-29 (55)	31-32-37 (47)
2500	-	-	-	-	2500	-	-	30-29-41 (44)	28-41-31 (48)

Figure 9: Full Elo-supporting match outcomes for SOLIS Mini with anchored advantage-vector planning against Stockfish. Each per-depth block reports W-D-L counts with match score in parentheses out of 100.